

Tackling Group Theory computationally with GAP

Omar Hatem

The American University in Cairo

November 26, 2025

Table of Contents

Introduction

GAP as a Programming Language

Looking at Groups in GAP

Testing Conjectures

Finding Trends

Advanced Topics

Table of Contents

Introduction

GAP as a Programming Language

Looking at Groups in GAP

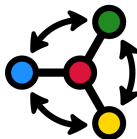
Testing Conjectures

Finding Trends

Advanced Topics

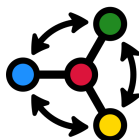
History

- ▶ The Groups, Algorithms, Programming (GAP) system is a computer algebra system with a focus on computational group theory.



History

- ▶ The Groups, Algorithms, Programming (GAP) system is a computer algebra system with a focus on computational group theory.
- ▶ GAP was started at Lehrstuhl D für Mathematik, RWTH Aachen in 1986.



RWTHAACHEN
UNIVERSITY

Present

- ▶ GAP is still going strong (v4.15.1) and has a large community.

Present

- ▶ GAP is still going strong (v4.15.1) and has a large community.
- ▶ It has a yearly conference (This year in Portugal)

Present

- ▶ GAP is still going strong (v4.15.1) and has a large community.
- ▶ It has a yearly conference (This year in Portugal)
- ▶ It also has an yearly excellence awards!

What does GAP do?

- ▶ Perform computations in group theory, ring theory, and other algebraic structures.

What does GAP do?

- ▶ Perform computations in group theory, ring theory, and other algebraic structures.
- ▶ Provide a rich library of functions for algebraic computations.

What does GAP do?

- ▶ Perform computations in group theory, ring theory, and other algebraic structures.
- ▶ Provide a rich library of functions for algebraic computations.
- ▶ Support for programming and scripting to automate tasks.

What does GAP do?

- ▶ Perform computations in group theory, ring theory, and other algebraic structures.
- ▶ Provide a rich library of functions for algebraic computations.
- ▶ Support for programming and scripting to automate tasks.
- ▶ Extensive documentation (gap-system.org/doc) and a supportive community.

Why GAP & computation?

- ▶ Interesting groups can get BIG (Monster Group $\approx 8.08 \times 10^{53}$).

Why GAP & computation?

- ▶ Interesting groups can get BIG (Monster Group $\approx 8.08 \times 10^{53}$).
- ▶ Number of groups can get BIG (non-isomorphic groups of order ≤ 2000 (423 million groups!)).

Why GAP & computation?

- ▶ Interesting groups can get BIG (Monster Group $\approx 8.08 \times 10^{53}$).
- ▶ Number of groups can get BIG (non-isomorphic groups of order ≤ 2000 (423 million groups!)).
- ▶ Can and has helped uncover patterns not easily seen at first.

What we're going to do today?

We're going to try to solve three group theory problems using GAP:

1. Given a group, explore what we can about it.

What we're going to do today?

We're going to try to solve three group theory problems using GAP:

1. Given a group, explore what we can about it.
2. Prove/Disprove a claim about automorphism groups.

What we're going to do today?

We're going to try to solve three group theory problems using GAP:

1. Given a group, explore what we can about it.
2. Prove/Disprove a claim about automorphism groups.
3. Conjecture trends about the commutativity ratio.

Online setup

We will be using mybinder to run GAP in an online Jupyter notebook. For a local setup, please refer to the GAP installation page, but we warn this might take time.

<https://hub.gesis.mybinder.org/user/omarhatemsalem-commutators-gap-xwx43q4r/tree>

Table of Contents

Introduction

GAP as a Programming Language

Looking at Groups in GAP

Testing Conjectures

Finding Trends

Advanced Topics

GAP is Interpreted

- ▶ GAP is an *interpreted* language - code is executed line by line without compilation

GAP is Interpreted

- ▶ GAP is an *interpreted* language - code is executed line by line without compilation
- ▶ This makes it ideal for interactive exploration and rapid prototyping

GAP is Interpreted

- ▶ GAP is an *interpreted* language - code is executed line by line without compilation
- ▶ This makes it ideal for interactive exploration and rapid prototyping
- ▶ You can type commands directly and see results immediately

Basic Syntax

GAP syntax is similar to many programming languages:

- ▶ Statements end with semicolons ; (suppress output with ;;)

Basic Syntax

GAP syntax is similar to many programming languages:

- ▶ Statements end with semicolons ; (suppress output with ;;)
- ▶ Comments start with #

Basic Syntax

GAP syntax is similar to many programming languages:

- ▶ Statements end with semicolons ; (suppress output with ;;)
- ▶ Comments start with #
- ▶ Assignment uses := (not =)

Basic Syntax

GAP syntax is similar to many programming languages:

- ▶ Statements end with semicolons ; (suppress output with ;;)
- ▶ Comments start with #
- ▶ Assignment uses := (not =)
- ▶ Comparison uses = for equality

Basic Syntax

GAP syntax is similar to many programming languages:

- ▶ Statements end with semicolons ; (suppress output with ;;)
- ▶ Comments start with #
- ▶ Assignment uses := (not =)
- ▶ Comparison uses = for equality
- ▶ Comparison uses <> for inequality

```
# This is a comment
```

```
x := 5;           # Assignment
```

```
y := x + 3;;      # No output
```

```
x = 5;           # Returns true
```

```
x <> 5;          # Returns false
```

Lists and Indexing

Lists are fundamental in GAP:

- ▶ Lists are written with square brackets: `[1, 2, 3]`

Lists and Indexing

Lists are fundamental in GAP:

- ▶ Lists are written with square brackets: `[1, 2, 3]`
- ▶ Indexing starts at **1** (not 0!)

Lists and Indexing

Lists are fundamental in GAP:

- ▶ Lists are written with square brackets: `[1, 2, 3]`
- ▶ Indexing starts at **1** (not 0!)
- ▶ Access elements with `list[i]`

Lists and Indexing

Lists are fundamental in GAP:

- ▶ Lists are written with square brackets: `[1, 2, 3]`
- ▶ Indexing starts at **1** (not 0!)
- ▶ Access elements with `list[i]`
- ▶ List ranges: `[1..10]` creates `[1, 2, ..., 10]`

```
mylist := [10, 20, 30];  
mylist[1];          # Returns 10  
mylist[2];          # Returns 20  
primes := [2, 3, 5, 7, 11];
```


Control Flow Keywords

GAP supports standard programming constructs:

▶ `if ... then ... elif ... else ... fi;`

Control Flow Keywords

GAP supports standard programming constructs:

- ▶ `if ... then ... elif ... else ... fi;`
- ▶ `for ... in ... do ... od;`

Control Flow Keywords

GAP supports standard programming constructs:

- ▶ `if ... then ... elif ... else ... fi;`
- ▶ `for ... in ... do ... od;`
- ▶ `while ... do ... od;`

Control Flow Keywords

GAP supports standard programming constructs:

- ▶ `if ... then ... elif ... else ... fi;`
- ▶ `for ... in ... do ... od;`
- ▶ `while ... do ... od;`
- ▶ `function (...) ... end;`

```
for i in [1..5] do
  if i mod 2 = 0 then
    Print(i, " is even\n");
  fi;
od;
```

Functions

Defining functions in GAP:

```
square := function(x)
  return x * x;
end;
```

```
square(5);  # Returns 25
```

Functions

Defining functions in GAP:

```
square := function(x)
  return x * x;
end;
```

```
square(5);  # Returns 25
```

Anonymous functions with `->`:

```
double := x -> 2 * x;
double(7);  # Returns 14
```

Your turn

Write a function that computes the n -th Fibonacci number.

Recall: The Fibonacci sequence is defined as:

- ▶ $F(0) = 0$
- ▶ $F(1) = 1$
- ▶ $F(n) = F(n-1) + F(n-2)$ for $n \geq 2$

Test your function with: `Fib(10)`

Table of Contents

Introduction

GAP as a Programming Language

Looking at Groups in GAP

Testing Conjectures

Finding Trends

Advanced Topics

Presentation of group

GAP provides built-in functions for a lot of known group types.
These include:

- ▶ $\mathbb{Z}_n$ as `CyclicGroup(n)`
- ▶ S_n as `SymmetricGroup(n)`
- ▶ A_n as `AlternatingGroup(n)`
- ▶ D_n as `DihedralGroup(2n)`
- ▶ Matrix Groups
- ▶ Quaternion Groups

Presentation of group

Otherwise, a group can be expressed through its *generators*.

Presentation of group

Otherwise, a group can be expressed through its *generators*.

```
G := Group((1,2,3));
```

Presentation of group

Otherwise, a group can be expressed through its *generators*.

```
G := Group((1,2,3));
```

```
G := Group((1,2,3), (4,3,5));
```

Presentation of group

Otherwise, a group can be expressed through its *generators*.

```
G := Group((1,2,3));
```

```
G := Group((1,2,3), (4,3,5));
```

Furthermore, any arbitrary group's elements will be represented by permutations (why?).

Analyzing a group

GAP can answer problems about the group and its elements. It can:

- ▶ List all the elements - `Elements(G)`
- ▶ List all the subgroups - `AllSubgroups(G)`
- ▶ List all the normal subgroups - `NormalSubgroups(G)`
- ▶ Get the order of an element - `Order(g)`

Analyzing a group

GAP can also check if a certain property holds for a group by prepending the property with the `Is` noun:

- ▶ `IsAbelian`
- ▶ `IsCyclic`
- ▶ `IsNormal`
- ▶ `IsIsomorphic`

Your turn

Choose some group and find the following properties:

- ▶ Order
- ▶ Number of Conjugacy classes
- ▶ Order of the Largest Proper Normal Subgroup
- ▶ Center
- ▶ Exponent (google what that is)
- ▶ Minimal Generating set

Table of Contents

Introduction

GAP as a Programming Language

Looking at Groups in GAP

Testing Conjectures

Finding Trends

Advanced Topics

Testing Conjectures

GAP provides an easy setting to check statements and conjectures. In this section, we want to check whether the following is true or false.

Is the automorphism group of a non-abelian group non-abelian?

Morphisms

In GAP, a morphism is a function between two objects of the same type (category).

$$\text{Set} \rightarrow \text{Set}$$
$$\text{Grp} \rightarrow \text{Grp}$$
$$\text{Ring} \rightarrow \text{Ring}$$

Group Homomorphisms

For groups, we can define a homomorphism in two ways:

1. Defining the image of the generators.
2. Returning a function that computes the elements.

Group Homomorphisms

For groups, we can define a homomorphism in two ways:

1. Defining the image of the generators.
2. Returning a function that computes the elements.

We also have `IsHomomorphism` to check if a function actually can be a homomorphism.

Group Automorphism

Automorphisms for groups follow the same rules as homomorphisms.

Group Automorphism

Automorphisms for groups follow the same rules as homomorphisms.

We also have `AutomorphismGroup(G)` to get the automorphism group.

Searching All Groups

GAP maintains a library of all groups up to certain orders. You can search through them using:

- ▶ `AllSmallGroups( $n$ )` - Returns all groups of order n
- ▶ `NumberSmallGroups( $n$ )` - Returns the number of groups of order n
- ▶ `SmallGroup( $n$ ,  $i$ )` - Returns the i -th group of order n

Searching All Groups

GAP maintains a library of all groups up to certain orders. You can search through them using:

- ▶ `AllSmallGroups(n)` - Returns all groups of order n
- ▶ `NumberSmallGroups(n)` - Returns the number of groups of order n
- ▶ `SmallGroup(n, i)` - Returns the i -th group of order n

You can also filter by properties:

- ▶ `AllSmallGroups(Size, n, IsAbelian, true)`
- ▶ `AllSmallGroups(Size, n, IsCyclic, false)`

Your turn

Check our claim again.

Is the automorphism group of a non-abelian group non-abelian?

Table of Contents

Introduction

GAP as a Programming Language

Looking at Groups in GAP

Testing Conjectures

Finding Trends

Advanced Topics

Finding Trends

GAP can also be great at analyzing trends and ideas.

Commutativity Ratio

For a finite group G the probability that two group elements chosen at random with replacement commute, which we denote by $\text{Pr}(G)$, is given by

$$\text{Pr}(G) = \frac{|\{(a, b) \in G \times G \mid ab = ba\}|}{|G|^2}$$

We call $\text{Pr}(G)$ the *commutativity ratio* of G .

Your turn

- ▶ Compute $\text{Pr}(G)$ for dihedral groups of order 6, 8, and 12
- ▶ Look for patterns in the results
- ▶ Can you conjecture a formula for $\text{Pr}(G)$ in terms of the number of conjugacy classes?

Your turn

- ▶ Compute $\text{Pr}(G)$ for dihedral groups of order 6, 8, and 12
- ▶ Look for patterns in the results
- ▶ Can you conjecture a formula for $\text{Pr}(G)$ in terms of the number of conjugacy classes?

Hint: The number of conjugacy classes can be found using `NrConjugacyClasses(G)`. Try to use `SmallGroups` to help you.

Table of Contents

Introduction

GAP as a Programming Language

Looking at Groups in GAP

Testing Conjectures

Finding Trends

Advanced Topics

Beyond the Basics

GAP offers many advanced features:

Character Theory:

- ▶ `CharacterTable(G)`, `Irr(G)`, `LinearCharacters(G)`

Beyond the Basics

GAP offers many advanced features:

Character Theory:

- ▶ `CharacterTable(G)`, `Irr(G)`, `LinearCharacters(G)`

Algebraic Categories:

- ▶ `IsMagma`, `IsSemigroup`, `IsMonoid`, `IsRing`, `IsField`

Beyond the Basics

GAP offers many advanced features:

Character Theory:

- ▶ `CharacterTable(G)`, `Irr(G)`, `LinearCharacters(G)`

Algebraic Categories:

- ▶ `IsMagma`, `IsSemigroup`, `IsMonoid`, `IsRing`, `IsField`

Free Presentations:

- ▶ Define groups by generators and relations using `FreeGroup`
- ▶ Example: $D_8 = \langle r, s \mid r^4 = s^2 = 1, srs = r^{-1} \rangle$

GAP in Research

GAP has been instrumental in many groundbreaking mathematical discoveries:

- ▶ Classification of groups of small order

GAP in Research

GAP has been instrumental in many groundbreaking mathematical discoveries:

- ▶ Classification of groups of small order
- ▶ Character theory computations for sporadic simple groups

GAP in Research

GAP has been instrumental in many groundbreaking mathematical discoveries:

- ▶ Classification of groups of small order
- ▶ Character theory computations for sporadic simple groups
- ▶ Verification of algebraic structures in representation theory

GAP in Research

GAP has been instrumental in many groundbreaking mathematical discoveries:

- ▶ Classification of groups of small order
- ▶ Character theory computations for sporadic simple groups
- ▶ Verification of algebraic structures in representation theory
- ▶ **Ore Conjecture (2010)**: Proved that every element of every finite non-abelian simple group is a commutator. GAP was used extensively for computational verification across thousands of groups.

Over 3000+ papers cite GAP in their research!

Further Exploration

Want to practice more with GAP?

- ▶ Joseph Gallian's *Contemporary Abstract Algebra* includes computer exercises designed for GAP

Further Exploration

Want to practice more with GAP?

- ▶ Joseph Gallian's *Contemporary Abstract Algebra* includes computer exercises designed for GAP
- ▶ Various teaching material on the GAP website

Further Exploration

Want to practice more with GAP?

- ▶ Joseph Gallian's *Contemporary Abstract Algebra* includes computer exercises designed for GAP
- ▶ Various teaching material on the GAP website
- ▶ Your CURIOSITY!

Thank You!

Questions?

Resources: gap-system.org